TIVIDY: Final Project Report

COS 214, 2026. Team Se7en
Repository: github.com/UncleShak/cos214-prac6-tividy. Report state matches branch integration-Code at commit c06f89a (1 November 2026).

1. Team information

Sin	Member	Student number	Git identity	Main responsibility
Envy	Onalerena Mojela	u25021002	Ona / onalerenamojela	Dependencies, routing decisions, progression, Visitor, Iterator, pattern audit
Wrath	Leul Tamiru	u25270665	pr1nc34	Approvals, rejection and escalation (Chain of Responsibility)
Lust	Shakir Alexander	u25122208	Shakir / UncleShak	Interface: TUI, plain mode, EngineGateway, LineInterpreter, integration
Pride	Frank Yambayamba	u25076427	Frank-Yambayamba	Work item lifecycle and State pattern, core engine
Sloth	Thakasa Ngcobo	u25556526	Thakasa	Events (Observer), history, access control (Proxy)
Gluttony	N'des Junior Lungwangu	u25069366	ndes-stack	Optional behaviour (Decorator), user actions with undo (Command), clock
Greed	Kadin Raju	u25175956	u25175956-ui	Participants and assignment,

Sin	Member	Student number	Git identity	Main responsibility
				Builder, Abstract Factory, Adapter, industry scenarios

Contribution statement

Each member owned one area of the engine (one "sin"), wrote the code, tests and UML for that area on their own branch, and merged it through a pull request into the integration branches. Shared work (fixing the integration build, the demo bug list, Docker, this report) was split as below.

Member	Commits (all branches)	Contribution
Shakir Alexander	26	Built the interactive interface (TUI and --plain mode), the real EngineGateway, input validation and the line interpreter; led integration of all sins into integration-Code; demo scripts.
Kadin Raju	19	Participants, roles and assignment rules; WorkflowBuilder/ScenarioBuilder; IndustryFactory; the ArkhamExpress and Dune Harvest scenarios; the warehouse Adapter.
N'des Junior Lungwangu	19	Decorators (Deadline, Retry, Validation, Audit), the Command family with CommandLog undo, ManualClock; fixed demo bug 4 (escalating held work) and bugs 2, 7 and 20; Docker setup and guide.
Frank Yambayamba	12	State pattern implementation, including the State interface and concrete state classes; core engine bug fixes and improvements; interface development and integration; UML diagram

Member	Commits (all branches)	Contribution
Leul Tamiru	11	Approval chain (ApprovalHandler, RoleBasedApprover, AutoApprover, ApprovalDesk), rejection and escalation.
Thakasa Ngcobo	10	Observer (EventSource, EventListener, monitoring and notification observers), InstanceHistory, AccessControlProxy with role and action policies.
Onalerena Mojela	10	Routing rules, DependencyGraph, ProgressionTracker, KeyValuePayload, Visitor reports, Iterators, pattern audit and decision log.

2. Research

Our research (Practical 6, Task 1) looked at how real workflow systems (Jira, Camunda, jBPM, ServiceNow, SAP Business Workflow, Microsoft Power Automate, Appian, Oracle HCM) handle the same problems TIVIDY must solve. The findings that shaped the design:

- **Definition vs instance.** A workflow definition is a reusable template; an instance is one independent run of it. Camunda deploys a definition once and runs many instances with their own state and variables (Camunda, n.d.-b). This led to separate WorkflowDefinition and WorkflowInstance classes, with each instance cloning its own copy of the work tree (Prototype).
- **Assignment and routing.** Work is assigned to a user, a queue or a role, sometimes conditionally on data such as priority or amount, and routed sequentially, in parallel, conditionally, or by escalation (Oracle, n.d.). This led to swappable AssignmentRule and RoutingRule strategies.
- **Roles, not names.** Camunda's candidate groups let anyone with a role claim a task (Camunda, n.d.-a). TIVIDY assigns by Role, not by hard-coded people.
- **Configurable approvals.** ServiceNow supports single, group and majority approvals and escalates to a backup approver after a deadline (ServiceNow, n.d.). TIVIDY uses a chain of approval handlers instead of one hard-coded check.
- **Events as first-class objects.** BPMN defines timer, message and error events that can start, interrupt or redirect a process (OMG, 2013). TIVIDY's Observer events (for example WORM_SIGN) can interrupt running work.
- **Integration through connectors.** Power Automate treats each external service as a swappable connector (Microsoft, n.d.). TIVIDY calls external systems through a

Connector interface with Adapters behind it.
- **History separate from live state.** SAP keeps an append-only work item history apart from live data (SAP, n.d.). TIVIDY records InstanceHistory per instance and uses snapshots for undo.
- **Monitoring as a separate layer.** Camunda Optimize is a separate product from the engine (Camunda, n.d.-c). TIVIDY's reports are Visitors that walk the work tree without changing work item classes.
- **Nested sub-processes.** BPMN call activities and embedded sub-processes (OMG, 2013) led to the Composite structure of Stage and WorkItem.
- **Reusability.** jBPM and Camunda run any process definition without changing engine code (Red Hat, n.d.); Appian lets non-developers change routing and approval rules (Appian, n.d.). TIVIDY's engine contains no industry names; industries plug in through factories and builders.

Assumptions

- The engine runs in one process with in-memory state; there is no database or network server.
- Time is simulated by a ManualClock that the user advances, so deadlines and escalation can be demonstrated and tested reliably.
- External systems (the warehouse) are simulated by in-process classes behind an Adapter.
- Participants are created in the engine; there is no login or authentication beyond role checks.
- Instances keep running against the definition they started with; definitions are not changed while instances run.

Boundaries

- TIVIDY manages the workflow process only. Industry details (order IDs, spice fields, stock levels) live in scenario code and per-instance data, never in the engine.
- The engine does not send real email, charge payments or call real web services.
- Reporting is text based (progress, audit and overdue reports); there is no graphical dashboard.
- There is no persistence across program runs.

3. The two industries

3.1 ArkhamExpress: order fulfilment and delivery (logistics)

ArkhamExpress processes customer orders and delivers packages. Participants are the order clerk, warehouse staff, the delivery manager and drivers.
Flow: ReceiveOrder → ValidateOrder → CheckStock → (out of stock) RequestStock from the external warehouse → PreparePackage → SelectDeliveryType → AssignDriver → Dispatch → Deliver. If delivery fails, ReturnToWarehouse opens and the order ends as *returned*, with the reserved stock given back.
Outcomes: delivered, cancelled, returned, failed.

Patterns it exercises: data-based routing on orderValid, inStock and delivered; the WarehouseConnector Adapter over WarehouseAPI; role-based assignment; Command undo (undoing a return takes the stock again).
Failures handled: invalid order, missing stock, failed delivery, rejection by the person doing the work, escalation of overdue work.

3.2 Dune Harvest: spice harvesting on Arrakis (mining)

One harvest run on a spice field. Participants include surveyors, harvester and carryall crews and an operations manager.
Flow: SurveyField → ApproveHarvestPermit (decided by the approval chain on estimatedYieldTonnes) → if the permit is granted, DeployHarvester and DeployCarryall run **in parallel** → Harvest (waits for both) → TransportToRefinery. If the permit is refused, AbandonRun.
Special behaviour: a WORM_SIGN event (Observer interrupt) cancels the deployment and harvest if they are still open and opens Evacuate, which must be done within 3 clock units (DeadlineDecorator). Harvest is audited (AuditDecorator). Only an OperationsManager may cancel work by hand (AccessControlProxy policy).
The two industries share no engine code. Both are installed through WorkflowEngine::installIndustry(IndustryFactory&) and listed in IndustryCatalog.

4. Updated requirements

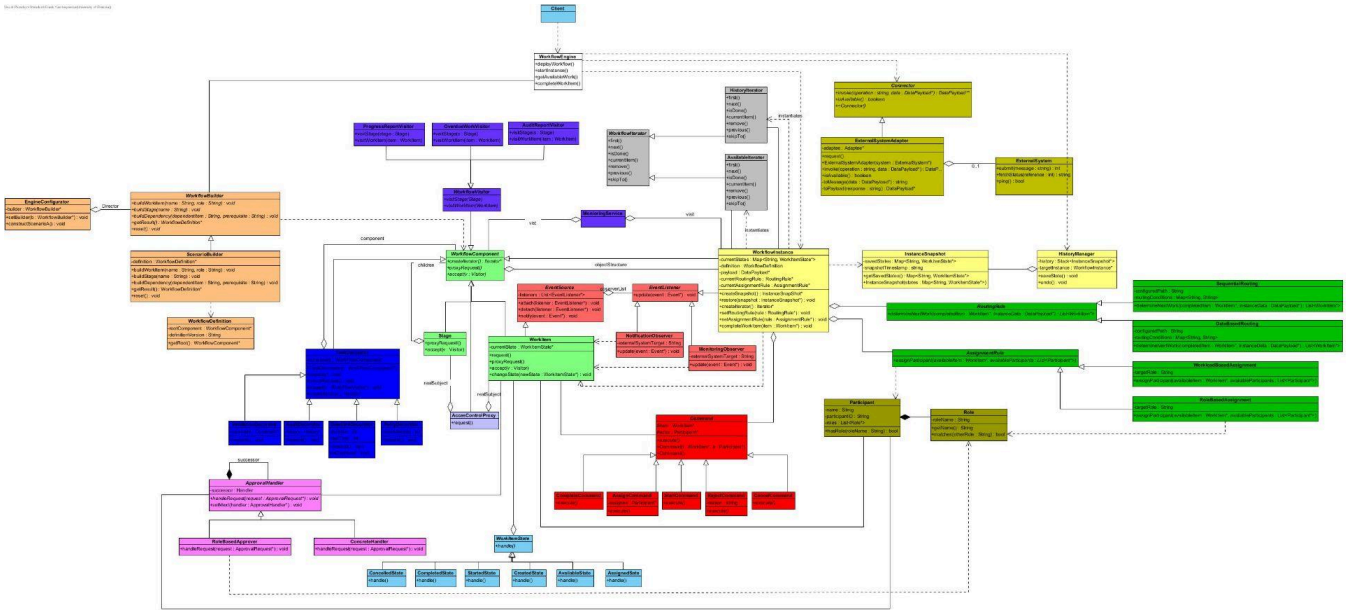
ID	Requirement
R1	Store reusable workflow definitions and start many independent instances from one definition.
R2	Represent work as work items, grouped into stages that can nest (sub-processes).
R3	Enforce dependencies between work items; support sequential and parallel work with joins.
R4	Route to the next work using configurable rules, including decisions based on instance data.
R5	Assign work to participants by role or workload, with configurable rules.
R6	Manage each work item's lifecycle: Created, Available, Assigned, Started, Completed, Rejected, Cancelled, Escalated, rejecting illegal actions.
R7	Support approvals through a configurable chain, plus rejection and escalation of overdue

ID	Requirement
	work.
R8	Publish events and let listeners react, including interrupting running work.
R9	Call external systems through a generic interface.
R10	Record per-instance history and support undo of user actions.
R11	Add optional behaviour (deadlines, retries, validation, auditing) to work items without changing them.
R12	Restrict actions by role (access control).
R13	Produce progress, audit and overdue reports, and iterate over available work and history.
R14	Run at least two different industries on the same engine with no industry code in the engine.
R15	Provide an interactive interface (TUI and plain typed commands) that talks to the engine through one facade.

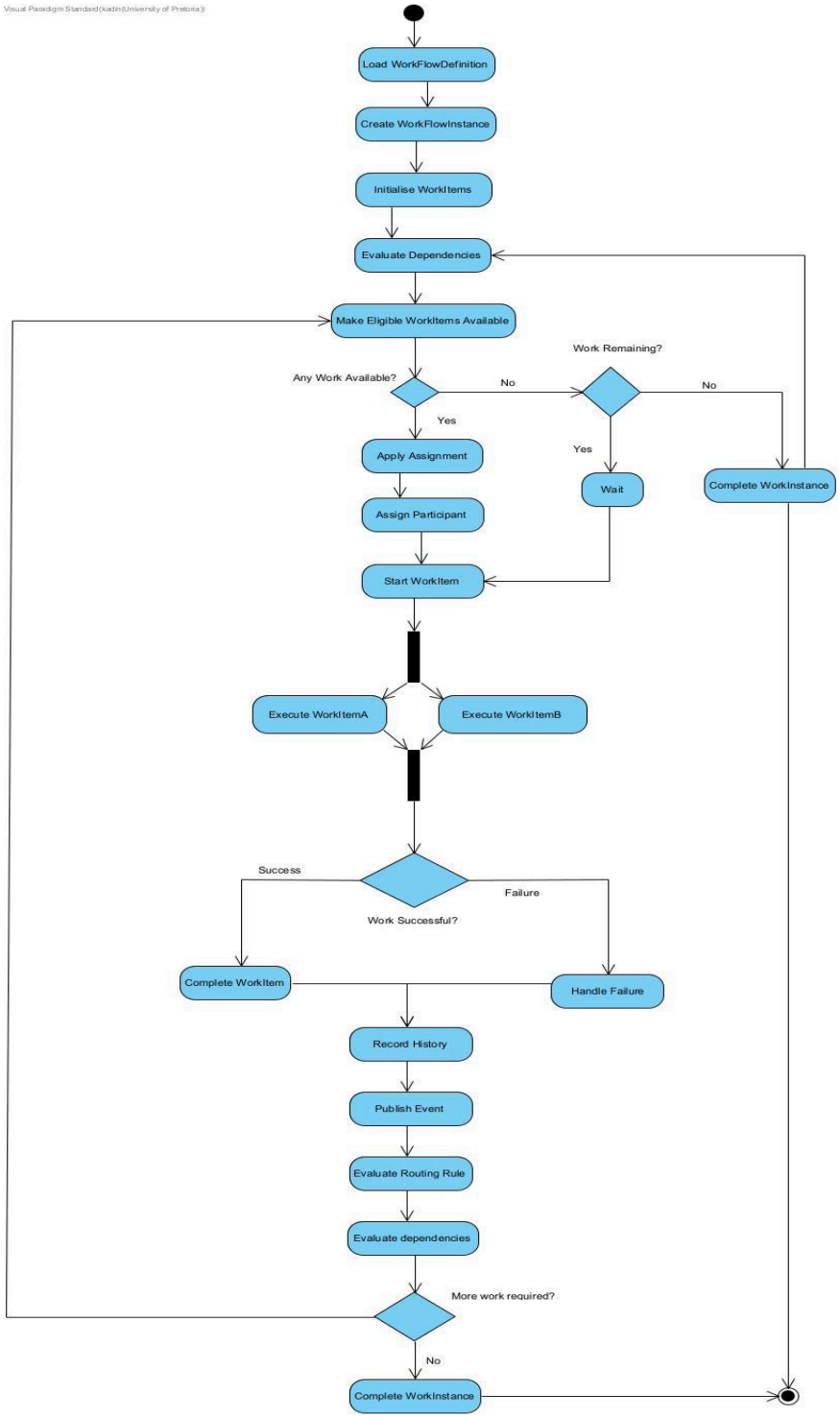
5. UML

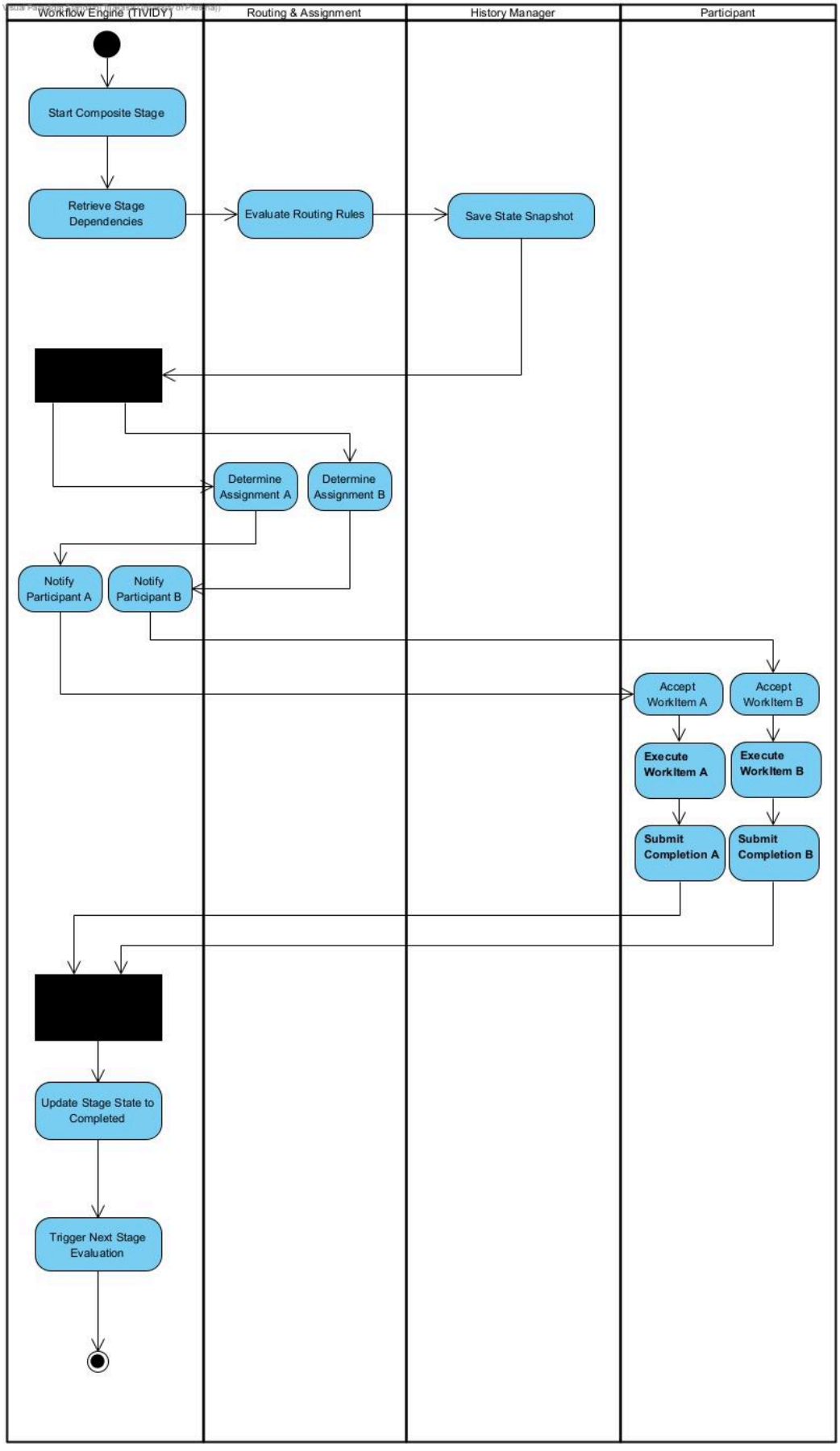
All diagrams are in docs/diagrams/ (images) and docs/vpp/ (Visual Paradigm sources).

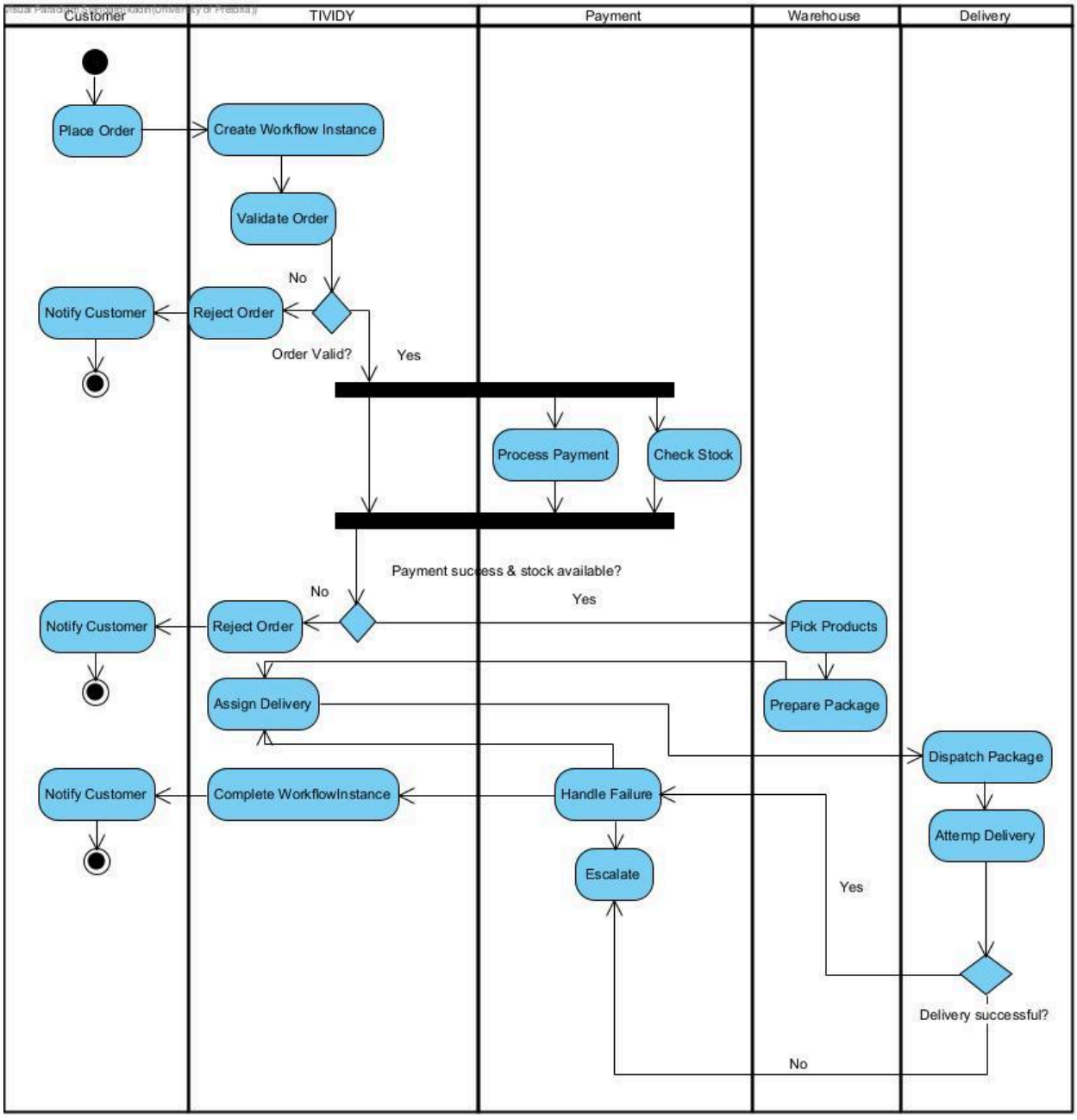
- **Class diagram:** ClassDiagram.jpg (source ClassDiagram.vpp). Includes the Decorator family (TaskDecorator, DeadlineDecorator, RetryDecorator, ValidationDecorator, AuditDecorator), the Command family with CommandLog, and Clock/ManualClock.



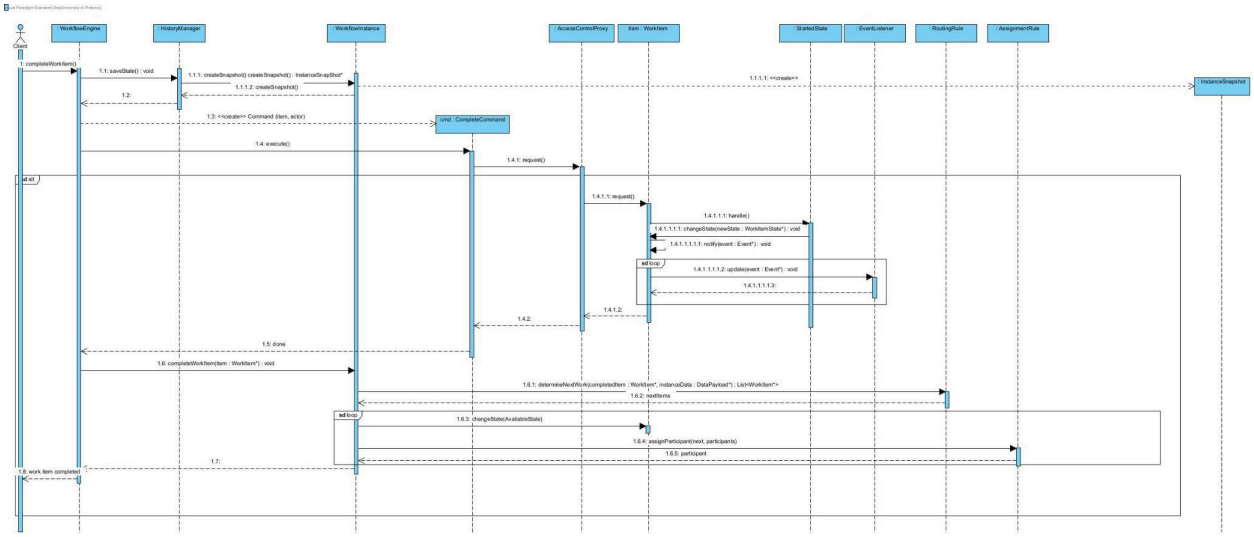
- **Activity diagrams:** Activity Diagram1.jpg, ActivityDiagram2.jpg (event dispatch and observer notification), AD3 _ WorkFlow.jpg.



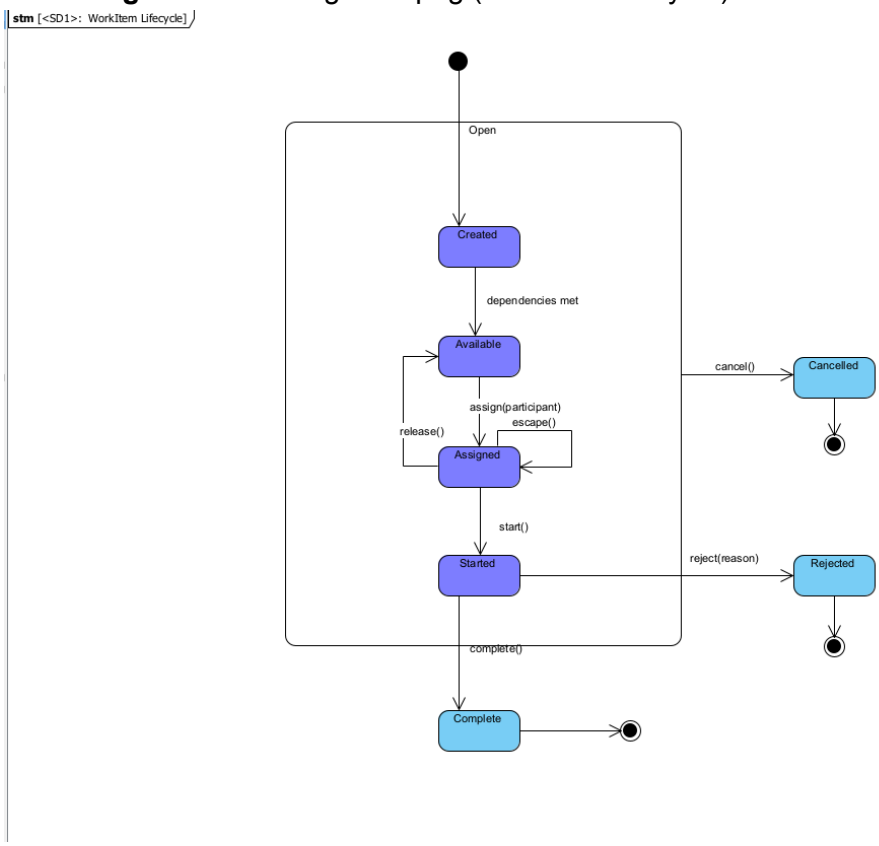

Workflow Engine (TIVIDY)
Routing & Assignment
History Manager
Participant
Start Composite Stage
Retrieve Stage Dependencies
Evaluate Routing Rules
Save State Snapshot
Determine Assignment A
Determine Assignment B
Notify Participant A
Notify Participant B
Accept WorkItem A
Accept WorkItem B
Execute WorkItem A
Execute WorkItem B
Submit Completion A
Submit Completion B
Update Stage State to Completed
Trigger Next Stage Evaluation


Customer
TIVIDY
Payment
Warehouse
Delivery
Place Order
Create Workflow Instance
Validate Order
No
Notify Customer
Reject Order
Order Valid?
Yes
Process Payment
Check Stock
Payment success & stock available?
No
Yes
Notify Customer
Reject Order
Pick Products
Assign Delivery
Prepare Package
Dispatch Package
Notify Customer
Complete WorkflowInstance
Handle Failure
Attemp Delivery
Escalate
Yes
Delivery successful?
No

- **Sequence diagram:** Sequence Diagram 1.jpg.



- **State diagram:** StateDiagram1.png (work item lifecycle).



-

6. Design patterns: roles and justifications

The spec asks for at least 10 distinct patterns that execute, at least 2 per category and at least 5 from the listed set. TIVIDY uses **16**: 3 creational, 5 structural and 8 behavioural, with **9** from the listed set (Observer, Iterator, Command, Adapter, Chain of Responsibility, Builder, Facade, Visitor, Proxy).

Pattern	Type	Participants in TIVIDY	Why we used it
Builder	Creational	Builder WorkflowBuilder/ScenarioBuilder; ConcreteBuilders ArkhamExpressBuilder, DuneHarvestBuilder; Director EngineConfigurator	A workflow definition is built in steps (structure, rules, connectors). The same steps build different industries.
Abstract Factory	Creational	AbstractFactory IndustryFactory; ConcreteFactories ArkhamExpressFactory, DuneHarvestFactory	Each industry needs a matching family of objects (definition, participants, starting data, connectors). The engine installs any industry without knowing its classes.
Prototype	Creational	Prototype WorkflowComponent::clone(); ConcretePrototypes WorkItem, Stage; Client WorkflowInstance	Each instance clones the definition's tree, so instances never share work items or state (definition vs instance, from our research).
Composite	Structural	Component WorkflowComponent; Leaf WorkItem; Composite Stage	Stages contain work items and other stages (sub-processes); the engine, visitors and decorators treat both the same way.
Decorator	Structural	Component WorkflowComponent; Decorator TaskDecorator; ConcreteDecorators DeadlineDecorator,	Optional behaviour varies per industry and per item. Decorators add it at run time, in any combination, without

Pattern	Type	Participants in TIVIDY	Why we used it
		RetryDecorator, ValidationDecorator, AuditDecorator	subclassing WorkItem.
Adapter	Structural	Target Connector; Adapter WarehouseConnector; Adaptee WarehouseAPI	External systems have their own interfaces. The engine only sees Connector.
Proxy	Structural	Subject WorkflowComponent; Proxy AccessControlProxy; policies RoleAccessPolicy, ActionAccessPolicy	Access control (only an OperationsManager may cancel) is checked in front of the real item, keeping the item free of security code.
Facade	Structural	Facade WorkflowEngine (and EngineGateway for the interface)	One simple entry point (start instance, assign, start, complete, reject, undo, advance clock) hides about 20 subsystems from the interface and the tests.
State	Behavioural	Context WorkItem; State WorkItemState; ConcreteStates Created, Available, Assigned, Started, Completed, Rejected, Cancelled, Escalated; StateFactory	Each state decides which actions are legal, so illegal actions throw instead of corrupting the item.
Strategy	Behavioural	Routing: RoutingRule → SequentialRouting, DataBasedRouting. Assignment: AssignmentRule → RoleBasedAssignment, WorkloadBasedAssignment	Routing and assignment rules vary most between organisations (Appian, n.d.), so they are swappable objects.

Pattern	Type	Participants in TIVIDY	Why we used it
Observer	Behavioural	Subject EventSource; Observer EventListener; ConcreteObservers MonitoringObserver, NotificationObserver, InterruptListener, InstanceHistory	State changes and events reach monitors, history and interrupt handlers without WorkItem knowing them.
Chain of Responsibility	Behavioural	Handler ApprovalHandler; ConcreteHandlers RoleBasedApprover, AutoApprover; Client ApprovalDesk; Request ApprovalRequest	Approval levels depend on value and role; a request passes up the chain until a level can decide it, and never goes unhandled.
Memento	Behavioural	Memento InstanceSnapshot; Originator WorkflowInstance; Caretaker CommandLog/History Manager	Undo restores an instance's exact earlier state without exposing its internals.
Command	Behavioural	Command Command; ConcreteCommands Assign, Start, Complete, Reject, Cancel, Escalate (Commands.h); Invoker CommandLog; Receiver WorkItem	Each user action becomes an object that can be logged and undone, and the interface's WorkAction maps one to one onto them.
Visitor	Behavioural	Visitor WorkflowVisitor; ConcreteVisitors ProgressReportVisitor, AuditReportVisitor, OverdueWorkVisitor; Elements WorkItem, Stage	New reports are added without changing the Composite classes (reporting as a separate layer, Camunda, n.d.-c).
Iterator	Behavioural	Iterator WorkflowIterator;	Clients walk available work or history without

Pattern	Type	Participants in TIVIDY	Why we used it
		ConcreteIterators AvailableWorkIterator, HistoryIterator; Aggregate ProgressionTracker	knowing how it is stored.

7. Implementation

The engine is C++11, built with CMake into the static library tividy_engine. Code is in code/, one folder per concern: workflow (Composite, definition, instance, snapshot), state, routing, progression, assignment, participants, approval, decorator, command, observer, history, proxy, visitor, iterator, integration, builder, scenarios (Arkham and Dune), core (the WorkflowEngine facade and WorkflowException), interface (TUI, plain mode, gateway) and app (the tividy program).

How one action flows through the system

1. The user picks an action in the TUI or types it in plain mode. LineInterpreter/InterfaceController validate the input and call EngineGateway.
2. The gateway calls the WorkflowEngine facade, which wraps the action in a Command and runs it through CommandLog, taking an InstanceSnapshot first for undo.
3. If an AccessControlProxy wraps the item, it checks the actor's role first.
4. The work item's current State accepts or rejects the action and moves to its next state.
5. The item notifies listeners (Observer): history records the change, monitors update, interrupt listeners may cancel or open work.
6. ProgressionTracker uses the DependencyGraph and the definition's RoutingRule to make the next work items Available, and checks whether the instance has finished with an outcome.
7. Decorators add their behaviour around the item, such as deadlines checked against the ManualClock. Advancing the clock escalates overdue work.

8. Decision history

ID	Date	Owner	Question	Decision
D-04	26 Oct	Envy	What should a routing rule receive and return, given one rule serves many instances?	Rules are stateless and read per-instance data, so one rule object can serve all instances.
D-05	26 Oct	Envy	Where does "what is available next, and is the	In a ProgressionTracker owned by each

ID	Date	Owner	Question	Decision
			instance finished?" live?	WorkflowInstance; snapshots also hold its progress.
D-06	26 Oct	Envy	How are dependencies represented and enforced?	A DependencyGraph of explicit prerequisite links; an item becomes Available only when all prerequisites are done.
D-07	26 Oct	Envy	How are decision branches expressed without an if/else chain in the engine?	DataBasedRouting with RoutingConditions (Strategy).
D-09/D-10	28 Oct	Envy	How does per-instance data and progression fit the team's DataPayload, WorkItem and WorkItemState?	KeyValuePayload extends DataPayload; progression uses the shared state names.
D-11/D-12	30 Oct	Envy	How are Visitor and Iterator built?	Visitors walk the Composite tree; ProgressionTracker is the Aggregate for Iterator.
D-S1	31 Oct	Sloth	How are state changes and events communicated without coupling WorkItem to observers?	EventSource/EventListener/Event (Observer).
D-S2	31 Oct	Sloth	How is history recorded per instance and kept	InstanceHistory realises both History and

ID	Date	Owner	Question	Decision
			compatible with AuditDecorator and HistoryIterator?	EventListener.
D-S3	31 Oct	Sloth	How is role-based access enforced without polluting core logic?	AccessControlProxy with pluggable AccessPolicy objects.
D-G1	Oct	Gluttony	How is optional behaviour added to only some items?	Decorators over WorkflowComponent, with time from an injected Clock so deadlines are testable.
D-G2	Oct	Gluttony	How are user actions undone?	Each action is a Command run through CommandLog, which stores a snapshot (Memento) before running it.
D-G3	1 Nov	Gluttony	Demo bug 4: overdue work that someone already holds was never escalated.	Assigned and Started may now move to Escalated; onClockAdvanced escalates held work; Escalate and Cancel use assigneeHoldsWork.
D-G4	1 Nov	Gluttony	Bugs 2, 7 and 20: returned parcels kept their stock, anyone could reject work, the clock could overflow.	Return gives stock back (and undo takes it again); only the assignee may reject, except the approval chain; the clock throws

ID	Date	Owner	Question	Decision
				on negative or overflowing advances and deadlines clamp to INT_MAX.
D-W1	30-Oct	Wrath	What does the approval chain pass around and how are deadlines handled?	Requests are encapsulated in an ApprovalRequest object, and handleRequest now takes ‘now : int’ so stateless handlers can evaluate time limits
D-W2	30 Oct	Wrath	Who starts the chain and manages the pending requests	Added ApprovalDesk as the client to own requests (replacing WorkItem), keeping pending states and deadlines isolated per instance
D-W3	30 Oct	Wrath	What do refused actions throw?	A unified WorkFlowException hierarchy (InvalidTransition, ValidationFailed, DeadlineExceeded, AccessDenied) so RetryDecorator never retries a refusal
D-W4D-G5	30 Oct	Wrath	Who owns the approval chain?	WorkFlowDefiniti on owns it; the chain is stateless so multiple workflow instances can share the exact same handler

ID	Date	Owner	Question	Decision
				objects
D-G5	30 Oct	Greed	How are workflow state changes represented without placing transition logic throughout the system?	The State pattern encapsulates state-specific behaviour in separate state classes, allowing WorkItems to change behaviour according to their current state.
D-G6	30 Oct	Greed	How can the system integrate an existing interface with the workflow without modifying the original implementation?	The Adapter pattern provides a compatible interface between the existing component and the workflow system, allowing them to work together without changing the adaptee.
D-G7	30 Oct	Greed	How are state-dependent operations integrated with the Command pattern?	Commands interact with the workflow through its public interface, allowing operations to trigger state transitions without directly depending on concrete state implementations.
D-P1	29 Oct	Pride	How do we guarantee strict data isolation between concurrent workflow	KeyValuePayload extends DataPayload; progression uses the shared state names.

ID	Date	Owner	Question	Decision
			instances?	Deep-copies the payload upon creation to preserve isolation.
D-P2	30 Oct	Pride	How are illegal lifecycle transitions handled without large switch/if-else blocks?	Implemented the State pattern for WorkItem. Concrete States enforce rules by throwing std::logic_error for invalid operations.
D-P3	30 Oct	Pride	How are stale deadlines handled when a user undoes an action?	Updated WorkflowEngine::undo() to immediately trigger an overdue check right after restoring a snapshot to re-evaluate deadlines against current engine time.

9. Traceability: requirements → classes → patterns → tests

Req	Main classes	Pattern	Tests
R1	WorkflowDefinition, WorkflowInstance, WorkflowComponent::clone	Prototype	tests/integration/test_engine.cpp, tests/lifecycle/test_lifecycle.cpp
R2	Stage, WorkItem	Composite	tests/envy/test_visitor.cpp, tests/integration/test_engine.cpp
R3	DependencyGraph, ProgressionTracker	(Iterator for available work)	tests/envy/test_dependencies.cpp, test_progression.cpp, test_cancel_dependants.cpp
R4	RoutingRule, SequentialRouting, DataBasedRouting	Strategy	tests/envy/test_routing.cpp, test_rejection_routing.cpp
R5	AssignmentRule, RoleBasedAssignment, WorkloadBasedAssignment, Participant, Role	Strategy	tests/greed/test_assignment.cpp, test_participants.cpp
R6	WorkItemState and the 8 concrete states	State	tests/lifecycle/test_lifecycle.cpp, tests/lust/test_engine_robustness.cpp
R7	ApprovalHandler, RoleBasedApprover, AutoApprover, ApprovalDesk	Chain of Responsibility	tests/lust/test_approvals.cpp, tests/envy/test_rejection_routing.cpp, tests/integration/test_gluttony_bugs.cpp (bug 7)
R8	EventSource, EventListener,	Observer	tests/sloth/test_sloth.cpp,

Req	Main classes	Pattern	Tests
	InterruptListener		tests/integration/test_scenarios.cpp
R9	Connector, WarehouseConnector, WarehouseAPI	Adapter	tests/greed/test_adapter.cpp, test_gluttony_bugs.cpp (bug 2)
R10	InstanceHistory, InstanceSnapshot, CommandLog, Commands	Memento, Command	tests/sloth/test_sloth.cpp, tests/envy/test_iterator.cpp, tests/integration/test_engine.cpp
R11	TaskDecorator and 4 concrete decorators, ManualClock	Decorator	tests/integration/test_scenarios.cpp, tests/envy/test_integration.cpp, test_gluttony_bugs.cpp (bug 20)
R12	AccessControlProxy, RoleAccessPolicy, ActionAccessPolicy	Proxy	tests/integration/test_proxy.cpp
R13	Report visitors, AvailableWorkIterator, HistoryIterator	Visitor, Iterator	tests/envy/test_visitor.cpp, test_iterator.cpp
R14	IndustryFactory, Arkham and Dune builders and factories, IndustryCatalog	Abstract Factory, Builder	tests/greed/test_builder.cpp, tests/integration/test_scenarios.cpp, tests/lust/test_scenarios.cpp, demo scripts
R15	WorkflowEngine, EngineGateway, InterfaceController, LineInterpreter	Facade	tests/gateway/test_real_engine.cpp, tests/lust/test_controller.cpp, test_interpreter.cpp, test_validation.cpp, test_layout.cpp

10. Build, run and test

Local (Linux or WSL)

1. Install tools: sudo apt install build-essential cmake libncurses-dev
2. Build: cmake -S . -B build then cmake --build build -j
3. Run the TUI: ./build/tividy (terminal at least 80 × 24; press q to quit)
4. Typed-command mode: ./build/tividy --plain
5. Demo script: ./build/tividy --plain --echo < tests/scripts/dune_demo.txt
6. All tests: ctest --test-dir build --output-on-failure
7. Clean build: rm -rf build, then repeat step 2.

Docker

1. docker build -t tividy . (builds everything and runs all tests; the build fails if a test fails)
2. TUI: docker run -it --rm tividy
3. Plain mode: docker run -it --rm tividy ./build/tividy --plain
4. Tests: docker run --rm tividy ctest --test-dir build --output-on-failure

Full guide with troubleshooting: docker.md.

Test results (integration-Code, c06f89a)

ctest suite	What it runs	Result
tividy_tests	All doctest files in tests/ except lust and gateway (envy, greed, integration, lifecycle, sloth)	187 test cases, 831 assertions, all pass
lust_tests	tests/lust (interface, interpreter, validation, approvals, robustness, scenarios)	38 test cases, 10 567 assertions, all pass
gateway_tests	The real gateway against the real engine	8 test cases, all pass
demo_arkham_demo	ArkhamExpress demo script through tividy --plain, compared with expected output	Pass
demo_dune_demo	Dune Harvest demo script, compared with expected output	Pass

11. Git and engineering evidence

- **Commits per member** (all remote branches): Shakir 26, Kadin 19, N'des 19, Frank 12, Leul 11, Thakasa 10, Onalerena 10. Every member has at least 10.
- **Closed pull requests:** #1 Greed, #2 Sloth, #3 Pride, #4 Envy, #5 Wrath, #6 Lust, #8 integration, and #12 to #17 (Envy, Wrath, integration-Code). That is well over the 3 required.
- **Branching:** one branch per sin, merged into integration-problem/integration-Code; the interface was developed on integration-ui. Work spans 29 September to 1 November 2026.
- **Continuous checking:** the Docker build runs every test, so a commit that breaks the build or a test cannot produce an image. Demo scripts are tests too, so the demo cannot silently break.
- **Bug process:** the team ran the demo, listed bugs, assigned them by sin and added a regression test for each fix (for example tests/integration/test_gluttony_bugs.cpp for bugs 2, 7 and 20).
- **Documentation:** Doxygen-style @brief comments in headers; decision logs in docs/envy and docs/sloth; pattern audit in docs/envy/patterns.md.

12. Bonus work

- A full-screen ncurses **TUI** plus a scriptable --plain mode.
- **Docker** image that builds and tests the whole system.
- Scripted demos that run as automated tests against expected output.
- 16 patterns (the minimum is 10), with 9 from the listed set (minimum 5).
- Undo for every user action (Command and Memento).
- A simulated clock with automatic escalation of overdue work.

13. References

Appian (n.d.) *Route Workflows Automatically*. Available at: https://docs.appian.com/suite/help/26.6/automate-routing.html (Accessed: 1 October 2026).

Atlassian (n.d.) *Understand workflows*. Available at: https://support.atlassian.com/jira-cloud-administration/docs/work-with-issue-workflows/ (Accessed: 1 October 2026).

Camunda (n.d.-a) *User tasks*. Available at: https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/ (Accessed: 1 October 2026).

Camunda (n.d.-b) *Versioning process definitions*. Available at: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/ (Accessed: 1 October 2026).

Camunda (n.d.-c) *Camunda Optimize reports*. Available at: https://camunda.com/platform/optimize/reports/ (Accessed: 1 October 2026).

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Microsoft (n.d.) *Connectors overview*. Available at: https://learn.microsoft.com/en-us/connectors/overview (Accessed: 1 October 2026).

Object Management Group (2013) *Business Process Model and Notation (BPMN), Version 2.0.2*. Available at: https://www.omg.org/spec/BPMN/2.0.2/PDF (Accessed: 1 October 2026).

Oracle (n.d.) *How workflow task routing is mapped out*. Available at: https://docs.oracle.com/en/cloud/saas/human-resources/faucf/how-workflow-task-routing-is-mapped-out.html (Accessed: 1 October 2026).

Red Hat (n.d.) *jBPM documentation*. Available at: https://docs.jboss.org/jbpm/release/7.42.0.Final/jbpm-docs/html_single/ (Accessed: 1 October 2026).

SAP (n.d.) *Workflow audit*. Available at: https://help.sap.com/docs/SAP_DISCLOSURE_MANAGEMENT/6fda503523d6413597f5cd81be59fa8a/495e9a5425504582a3d871cf5a9ec22b.html (Accessed: 1 October 2026).

ServiceNow (n.d.) *Simple approvals in workflow*. Available at: https://support.servicenow.com/kb?id=kb_article_view&sysparm_article=KB0538552 (Accessed: 1 October 2026).